QueueSmart Notification and History

Assignment 4 part 6 contribution handoff

Notification and participation history data now persists in SQLite through the existing SQLAlchemy database layer. This document covers part 6 and is intended to be combined with the other members' sections for the group submission.

GitHub repository

github.com/ramButTan-hw/COSC-4353-QueueSmart

Implemented behavior

• Saved participation history and per-user notifications, including joins, position changes, queue status changes, completions and staff alerts.

• Saved read, read-all and clear actions. The database stores sent or viewed; the existing UI continues to receive its read boolean. Clearing notifications preserves history.

• Preserved authenticated user isolation, UTC date filters, newest-first results and the existing API response format.

• Committed history and related notifications together. Failed database writes roll back activity and restore the in-memory queue state.

Database relationships and validation

Both tables use a sequence primary key and a unique public ID. History users and notification recipients reference user_credentials.id. Notifications reference participation_history.id. Foreign keys cascade on account deletion. Required fields, length limits, allowed statuses and timestamp checks are enforced by the database.

Service ID and service name are historical snapshots so completed history survives service deletion. Timestamps use UTC Unix milliseconds. The appendix contains the SQLite table and index statements generated from the implemented models.

Validation and contribution

Test and coverage evidence

371 backend tests passed. Statement coverage: 90.24% (1,063 of 1,178 statements), exceeding the 80% gate. The new activity persistence and table modules each have 100% statement coverage.

15 frontend and API integration tests passed. These include restarting an actual server process against the same SQLite file and retrieving saved history afterward.

The 26 new database-focused tests cover durable reads and writes, recipient isolation, read and clear actions, failed transactions, constraints, cascading deletion and interrupted visits. Verification used Python 3.14 with the existing frozen uv.lock dependencies on November 5, 2026.

Coverage is statement coverage, not branch coverage. The denominator includes the existing unused Wait-Time Estimation Logic.py file. The detailed text summary is in docs/a4-part6/coverage-summary.txt.

Contribution entry

Group member name: _____

Contribution: Implemented part 6 SQLAlchemy tables and validations, transactional notification and history persistence, durable read and clear actions, restart recovery, database tests and a process restart integration test. Prepared the SQL schema and coverage evidence.

Integration notes: Reused the team's database foundation and credentials foreign keys while preserving the A3 UI and API contracts. The queue persistence module must provide stable participation IDs and restore queue entries before startup recovery runs. Add actual team discussion notes to the final contribution table.

Remaining group integration

Service and queue runtime state remains in memory. Until queue persistence is integrated, startup marks orphaned waiting visits Removed and adds an interruption notice so users can rejoin. Run one backend worker. The current activity unit of work loads activity records to reuse the existing rules; larger datasets will need SQL filtering and pagination.

Review, commit and push this contribution under your own Git identity. Add the remaining tables and all four members' verified contributions, then submit one combined Word or PDF file to Canvas. This handoff does not represent completion of the entire group assignment.

History schema

SQLite DDL from the implemented SQLAlchemy models. The existing user_credentials table must be created first. Enable PRAGMA foreign_keys = ON for each SQLite connection.

```
-- A4 Part 6. Create user_credentials first; foreign keys must be enabled.

PRAGMA foreign_keys = ON;

CREATE TABLE participation_history (
  sequence INTEGER NOT NULL,
  id VARCHAR(100) NOT NULL,
  user_id VARCHAR(36) NOT NULL,
  service_id BIGINT NOT NULL,
  service_name VARCHAR(100) NOT NULL,
  joined_at BIGINT NOT NULL,
  completed_at BIGINT,
  outcome VARCHAR(10) NOT NULL,
  position BIGINT NOT NULL,
  close_notified BOOLEAN NOT NULL,
  next_notified BOOLEAN NOT NULL,
  queue_open BOOLEAN NOT NULL,
  CONSTRAINT pk_participation_history PRIMARY KEY (sequence),
  CONSTRAINT ck_participation_history_id_length CHECK (length(id) BETWEEN 1 AND 100),
  CONSTRAINT ck_participation_history_service_name_length CHECK (length(service_name)
    BETWEEN 1 AND 100),
  CONSTRAINT ck_participation_history_positive_numbers CHECK (service_id > 0 AND position
    > 0),
  CONSTRAINT ck_participation_history_join_time_valid CHECK (joined_at >= 0),
  CONSTRAINT ck_participation_history_outcome_valid CHECK (outcome IN ('Waiting',
    'Served', 'Left queue', 'Removed')),
  CONSTRAINT ck_participation_history_completion_valid CHECK ((outcome = 'Waiting' AND
    completed_at IS NULL) OR (outcome <> 'Waiting' AND completed_at >= joined_at AND
    completed_at IS NOT NULL)),
  CONSTRAINT ck_participation_history_flags_valid CHECK (close_notified IN (0, 1) AND
    next_notified IN (0, 1) AND queue_open IN (0, 1)),
  CONSTRAINT uq_participation_history_id UNIQUE (id),
  CONSTRAINT fk_participation_history_user_id_user_credentials FOREIGN KEY(user_id)
    REFERENCES user_credentials (id) ON DELETE CASCADE
);

CREATE INDEX ix_participation_history_user_id ON participation_history (user_id);
```

Notification schema

SQLite DDL from the implemented SQLAlchemy models. The existing user_credentials table must be created first. Enable PRAGMA foreign_keys = ON for each SQLite connection.

```
CREATE TABLE notification (
  sequence INTEGER NOT NULL,
  id VARCHAR(36) NOT NULL,
  recipient_id VARCHAR(36) NOT NULL,
  participation_id VARCHAR(100) NOT NULL,
  role VARCHAR(20) NOT NULL,
  kind VARCHAR(40) NOT NULL,
  message VARCHAR(1000) NOT NULL,
  type VARCHAR(10) NOT NULL,
  timestamp BIGINT NOT NULL,
  status VARCHAR(6) NOT NULL,
  CONSTRAINT pk_notification PRIMARY KEY (sequence),
  CONSTRAINT ck_notification_id_length CHECK (length(id) BETWEEN 1 AND 36),
  CONSTRAINT ck_notification_message_length CHECK (length(message) BETWEEN 1 AND 1000),
  CONSTRAINT ck_notification_kind_length CHECK (length(kind) BETWEEN 1 AND 40),
  CONSTRAINT ck_notification_role_valid CHECK (role IN ('student', 'staff')),
  CONSTRAINT ck_notification_type_valid CHECK (type IN ('success', 'warning', 'info',
    'error')),
  CONSTRAINT ck_notification_status_valid CHECK (status IN ('sent', 'viewed')),
  CONSTRAINT ck_notification_timestamp_valid CHECK (timestamp >= 0),
  CONSTRAINT uq_notification_id UNIQUE (id),
  CONSTRAINT fk_notification_recipient_id_user_credentials FOREIGN KEY(recipient_id)
    REFERENCES user_credentials (id) ON DELETE CASCADE,
  CONSTRAINT fk_notification_participation_id_participation_history FOREIGN
    KEY(participation_id) REFERENCES participation_history (id) ON DELETE CASCADE
);

CREATE INDEX ix_notification_recipient_id ON notification (recipient_id);
```